МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений

Студент гр. 4343 _____ Едина А.С.

Преподаватель _____ Пуеров Г. Ю.

Санкт-Петербург
2026

Цель работы

Изучение метода Гаусса с полным выбором ведущего элемента и метода простых итераций (метод Якоби) решения систем линейных алгебраических уравнений (СЛАУ), реализация соответствующих алгоритмов на языке C++, а также сравнительный анализ их точности, производительности на различных типах матриц.

Теоретическая часть

Метод Гаусса.

Рассматривается система линейных уравнений n-го порядка

$$\begin{gathered} a_{11}x_1 + a_{12}x_2 + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{2n}x_n = b_2 \\ . \quad . \quad . \quad . \quad . \\ a_{n1}x_1 + a_{n2}x_2 + a_{\mathrm{nn}}x_n = b_n, \end{gathered} \tag{2.1}$$

что в векторном виде записывается как $\mathrm{A}\mathrm{x} = b$.

Суть метода исключения по главным элементам (метод Гаусса) заключается в следующем. Находится наибольший по абсолютной величине коэффициент a_{kj}. Для исключения x_j из i-го уравнения $(i \neq k)$ необходимо умножить k-е уравнение на $a_{\mathrm{ij}}/a_{\mathrm{kj}}$ и вычесть его из i-го уравнения, после чего процесс повторяется для исключения другого неизвестного из оставшихся n-1 уравнений и т. д. В результате система (2.1) приводится к треугольному виду

$$\begin{gathered} x_1 + \alpha_{12}x_2 + \ldots + \alpha_{1n}x_n = \beta_n \\ x_2 + \ldots + \alpha_{2n}x_n = \beta_n \\ . \quad . \quad . \quad . \quad . \quad . \\ x_n = \beta_n, \end{gathered} \tag{2.2}$$

из которого легко находятся неизвестные $x_1,\ldots,x_n$. Процесс приведения системы к виду (2.2) называется прямым ходом, а нахождение неизвестных $x_1,\ldots,x_n$ - обратным ходом метода Гаусса.

Следует отметить, что если матрица заданной системы вырожденная, то перед исключением некоторой неизвестной главный элемент a_{kj} окажется равным нулю, что и будет свидетельствовать о равенстве нулю определителя системы.

$$\|r\|_2 = \sqrt{(r_1^2 + r_2^2 + \cdots + r_n^2)} = \sqrt{\left(\Sigma_{i=1}^{n} r_i^2\right)}$$

Данная величина показывает среднеквадратичную ошибку по всем уравнениям, а также она чувствительна к большим отклонениям(Евклидова норма)

Метод простых итераций

Данный итерационный метод основан на преобразовании исходной системы к эквивалентному виду $x = Bx + c$, где

- $B = -D^{-1}(L + U)$,
- $c = D^{-1}b$,
- D – диагональная часть матрицы A,
- L, U – содержат верхнюю и нижнюю треугольные части матрицы.

Формула пересчёта

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{\substack{j \neq i \\ j = 1}}^{n} a_{ij} x_j^{(k)}\right)$$

Элементы матрицы B:

- Для недиагональных элементов $i \neq j$:

$$(L + U)_{ij} = a_{ij}$$

так как L и U содержат все недиагональные элементы. Умножение на D^{-1} слева даёт

$$b_{ij} = \left(-D^{-1}(L + U)_{ij}\right) = -\frac{a_{ij}}{a_{ii}}$$

- Для $i = j$:

$$(L + U)_{ii} = 0$$

поэтому $b_{ii} = 0$

Для практической оценки сходимости используется строчная норма матрицы итераций $\|B\|_\infty$ (максимальная сумма модулей элементов строки), которая вычисляется как:

$$\|B\|_\infty = max\left(\sum_{j=1}^{n} |a_{ij}|\right)$$

- Если $\|B\|_\infty$<1 – сходимость метода гарантирована.
- Если норма близка к 1 – сходимость будет медленной.
- Если $\|B\|_\infty$≥1 – метод может расходиться.

Критерий остановки итераций

В реализованной программе итерационный процесс прекращается, когда евклидова норма изменения решения между двумя последовательными итерациями становится меньше заданной точности $tol = 10^{-12}$

Тестирование программы

Таблица 1 - Результаты тестирования метода Гаусса

№ п/п	Входные данные	Выходные данные
1.	./generate 10 1 test_10.txt сгенерирована случайная матрица с диагональным преобладанием данные записаны в test_10.txt root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./gauss test_10.txt 10 -219.9831738229 37.2887934519 - 46.9023282870 - 3.1026988708 2.6831608819 - 17.2222025661 - 13.9405379844 13.2537606556 20.1548561690 - 27.0251246789 - 44.8027045268 -3.6830907315 180.7452032417 5.7699176500 - 8.3124147266 - 40.5076082572 - 7.3613383255 - 4.5594298863 26.2090250967 46.7423241569 - 24.2916577489 5.7877898470 -40.9863557681 - 25.1748930485 - 239.0728862036 - 46.5285215103 - 2.0207690857 37.2626268928 - 13.2715714000 - 24.8207703751 20.1283924095 - 25.5339243595 23.9297529015 3.3770824767 42.0080519808 24.3358605638 -	размер матрицы: 10x10 решаем систему система успешно решена за 6.155e-06 сек. РЕЗУЛЬТАТЫ норма невязки L2: 1.865175e-14 норма невязки L_inf: 1.421085e-14 вектор решений x: x[0] = 0.208373 x[1] = -0.004619 x[2] = -0.124163 x[3] = -0.115853 x[4] = -0.138830 x[5] = 0.040496 x[6] = 0.138644 x[7] = 0.025520 x[8] = 0.134772 x[9] = 0.186003 размер матрицы: 10x10 решаем систему методом Eigen Eigen решил систему за 5.9763e-05 сек. результаты EIGEN норма невязки L2: 1.432145e-14 норма невязки L_inf: 7.105427e-15 Сравнение времени: Метод Гаусса: 59763 нс Метод Eigen: 61555 нс

	314.8483955553 - 12.5366756709 - 19.5403970972 42.9211029397 - 41.3766561869 - 46.6682140772 35.8017877714 40.1777588944 -25.9157912836 15.3802040823 39.7193501658 36.3319082751 - 277.8546829938 - 10.3888094168 42.6595971822 4.1930534046 26.1082369380 45.0380686873 41.4593333114 4.0340874404 36.0001099568 41.2295721592 16.5233570270 16.5798409460 259.0048209357 30.4816174340 44.5630477642 28.0883841800 12.5636803138 13.3134542019 -37.2777459690 - 14.5786389220 43.3346871258 23.2040715549 - 2.1938386185 - 27.6836598701 - 225.6647473217 - 8.6776646506 44.8101908739 - 13.8719424831 - 44.6351583401 38.8533934447 48.4277547305 - 44.0202721569 18.3413132786 - 39.4468205487 - 3.0142516250 - 7.3086459835 - 279.9918658841 4.8967691640 49.5275588170 18.2811076456	

	-22.6178603857 11.1741886718 14.9281829798 20.0473272498 42.2564688916 - 15.2869841682 22.9712449354 49.0831476539 - 269.7471302221 49.9794467996 - 38.0468791365 17.6187661499 20.1077640084 21.4313651103 - 46.4681823927 - 37.6865063301 - 12.7340333855 9.5706263623 5.8455134384 41.0371167734 - 249.3271383195 - 28.3516330632	
2.	Матрица размером 5000x5000 ./generate 5000 1 test_5000.txt сгенерирована случайная матрица с диагональным преобладанием данные записаны в test_5000.txt	./gauss test_5000.txt размер матрицы: 5000x5000 решаем систему система успешно решена за 28.0832 сек. РЕЗУЛЬТАТЫ норма невязки L2: 7.611070e-12 норма невязки L_inf: 4.973799e-13 (матрица слишком большая для вывода вектора решений) ./eigen_solver test_5000.txt размер матрицы: 5000x5000 решаем систему методом Eigen Eigen решил систему за 4.73188 сек. результаты EIGEN норма невязки L2: 3.477094e-12 норма невязки L_inf: 2.842171e-13 Сравнение времени: Метод Гаусса: 28 сек Метод Eigen: 5 сек
3.	Матрица Гильберта ./generate 10 2 test_g_10.txt сгенерирована матрица Гильберта данные записаны в test_g_10.txt	root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./gauss test_g_10.txt размер матрицы: 10x10 решаем систему система успешно решена за 1.584e-06 сек. РЕЗУЛЬТАТЫ

	10 1.0000000000 0.5000000000 0.3333333333 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 2.9289682540 0.5000000000 0.3333333333 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 2.0198773449 0.3333333333 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 1.6032106782 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 1.3468004218 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909	норма невязки L2: 3.845925e-16 норма невязки L_inf: 2.220446e-16 вектор решений x: x[0] = 0.999941 x[1] = 1.003910 x[2] = 0.937279 x[3] = 1.415873 x[4] = -0.364518 x[5] = 3.297939 x[6] = -0.709454 x[7] = 0.912663 x[8] = 1.850487 x[9] = 0.655828 ./eigen_solver test_g_10.txt размер матрицы: 10x10 решаем систему методом Eigen Eigen решил систему за 3.794e-06 сек. результаты EIGEN норма невязки L2: 7.447602e-16 норма невязки L_inf: 4.440892e-16 Сравнение времени: Метод Гаусса: 15844 нс Метод Eigen: 37944 нс

	0.0833333333 0.0769230769 0.0714285714 1.1682289932 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 1.0348956599 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 0.0625000000 0.9307289932 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 0.0625000000 0.0588235294 0.8466953798 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 0.0625000000 0.0588235294 0.0555555556 0.7772509353 0.1000000000 0.0909090909 0.0833333333 0.0769230769	

	0.0714285714 0.0666666667 0.0625000000 0.0588235294 0.0555555556 0.0526315789 0.7187714032	

Таблица 2 - Результаты тестирования метода простых итераций

№ п/п	Входные данные	Выходные данные
1	./generate 10 1 test1.txt сгенерирована случайная матрица с диагональным преобладанием данные записаны в test1.txt root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./jacobi test1.txt	размер матрицы: 10x10 решение системы методом Якоби (простых итераций) норма матрицы итераций Якоби = 0.978682 метод сошёлся за 27 итераций. время выполнения: 4.151e-05 сек. РЕЗУЛЬТАТЫ Норма невязки L2: 7.615469e-11 Норма невязки L_inf: 4.595627e-11 вектор решения x: x[0] = 0.22394351 x[1] = -0.17460753 x[2] = 0.00559111 x[3] = -0.05125753 x[4] = -0.01936527 x[5] = -0.37727272 x[6] = 0.07943855 x[7] = -0.10443357 x[8] = -0.09613220 x[9] = 0.22515691
2	./generate 10000 1 test1.txt сгенерирована случайная матрица с диагональным преобладанием данные записаны в test1.txt root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# time ./jacobi test1.txt	размер матрицы: 10000x10000 решение системы методом Якоби (простых итераций) норма матрицы итераций Якоби = 0.999996 метод сошёлся за 6 итераций. время выполнения: 3.72329 сек.

		РЕЗУЛЬТАТЫ Норма невязки L2: 8.009950e-11 Норма невязки L_inf: 3.124612e-12 (размер матрицы > 20, вывод решения опущен) первые 5 компонент: 8.332566e-05 -1.762987e-04 -1.725032e-04 -6.922942e-05 -1.345366e-04 real 0m40.495s user 0m23.385s sys 0m0.998s
3	./generate 10 2 test1.txt сгенерирована матрица Гильберта данные записаны в test1.txt root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./jacobi test1.txt	размер матрицы: 10x10 решение системы методом Якоби (простых итераций) норма матрицы итераций Якоби = 12.6567 предупреждение: ‖B‖ >= 1, метод может расходиться или сходиться очень медленно метод не сошёлся за 100000 итераций. время выполнения: 0.0512785 сек. РЕЗУЛЬТАТЫ Норма невязки L2: inf Норма невязки L_inf: inf вектор решения x: x[0] = -inf x[1] = -inf x[2] = -inf x[3] = -inf x[4] = -inf x[5] = -inf x[6] = -inf x[7] = -inf x[8] = -inf x[9] = -inf
4	./generate 300 2 test1.txt сгенерирована матрица Гильберта данные записаны в test1.txt	размер матрицы: 300x300

	root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./jacobi test1.txt	решение системы методом Якоби (простых итераций) норма матрицы итераций Якоби = 414.695 предупреждение: ‖B‖ >= 1, метод может расходиться или сходиться очень медленно метод не сошёлся за 100000 итераций. время выполнения: 35.2862 сек. РЕЗУЛЬТАТЫ Норма невязки L2: inf Норма невязки L_inf: inf (размер матрицы > 20, вывод решения опущен) первые 5 компонент: -inf -inf -inf -inf -inf

Сравним методы.

root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./generate 2000 1 test1.txt

сгенерирована случайная матрица с диагональным преобладанием

данные записаны в test1.txt

root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./full test1.txt

размер системы: 2000x2000

норма матрицы итераций Якоби = 0.999979661664

метод ГАУССА

Время: 15.776555579000 сек

Невязка L2 : 3.253836109774e-12

Невязка L_inf : 3.765876499529e-13

метод ЯКОБИ

Сошёлся за 7 итераций

Время: 1.674623640000e-01 сек

Невязка L2 : 2.553865291395e-10

Невязка L_inf : 2.417976929792e-11

метод EIGEN

Время: 1.802565296100e+01 сек

Невязка L2 : 1.148290318872e-12

Невязка L_inf : 1.136868377216e-13

ПОПАРНЫЕ РАЗНОСТИ РЕШЕНИЙ

||Gauss - Jacobi||_2 = 5.105864697516e-15

||Gauss - Jacobi||_inf = 4.784042086092e-16

||Gauss - Eigen||_2 = 5.563245742661e-17

||Gauss - Eigen||_inf = 6.613633252162e-18

||Jacobi - Eigen||_2 = 5.105231294845e-15

||Jacobi - Eigen||_inf = 4.779705277402e-16

Вывод

В ходе лабораторной работы реализованы и исследованы два метода решения СЛАУ: Гаусса с полным выбором ведущего элемента и метод простых итераций.

Эксперименты показали, что для диагонально преобладающей матрицы размером 2000×2000 методы дают близкие результаты: разности между решениями не превышают 10^{-15}, а невязки имеют порядок $10^{-10} - 10^{-13}$. Метод Якоби сошёлся всего за 7 итераций, показав наилучшее время (0.17 с), что значительно быстрее Гаусса (15.8 с). Норма матрицы итераций $\| B \| \infty = 0.99998$ оказалась меньше единицы, что подтвердило гарантированную сходимость.

На матрице Гильберта метод Якоби не сошёлся за 100000 итераций: норма $\| B \| = 414.7 > 1$ привела к расходимости и невязкам, равным бесконечности. Это демонстрирует, что метод Якоби эффективен только для систем с диагональным преобладанием, тогда как на плохо обусловленных матрицах он неприменим. Прямой метод (Гаусс) в таких случаях также теряет точность, но хотя бы дает численный результат.

Приложения А

Исходный код gauss.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include <cmath>
#include <chrono>
#include <iomanip>

using namespace std;

const int NMAX = 5000;

int GAUSS(const vector<vector<double>>& a_in, const vector<double>& b_in,
vector<double>& x, int n) {
    if (n < 2) return 2;
    if (n > NMAX) return 3;

    vector<vector<double>> a = a_in;
    vector<double> b = b_in;
    x.assign(n, 0.0);

    for (int i = 0; i < n; ++i) {

        int pivot_row = i;
        double max_val = std::abs(a[i][i]);

        for (int k = i + 1; k < n; ++k) {
            if (std::abs(a[k][i]) > max_val) {
                max_val = std::abs(a[k][i]);
                pivot_row = k;
            }
        }

        if (max_val < 1e-12) {
            return 1;
        }

        if (pivot_row != i) {
            swap(a[i], a[pivot_row]);
            swap(b[i], b[pivot_row]);
        }

        for (int k = i + 1; k < n; ++k) {
            double factor = a[k][i] / a[i][i];
            for (int j = i; j < n; ++j) {
                a[k][j] -= factor * a[i][j];
            }
            b[k] -= factor * b[i];
        }
    }

    for (int i = n - 1; i >= 0; --i) {
        double sum = 0.0;
        for (int j = i + 1; j < n; ++j) {
            sum += a[i][j] * x[j];
        }
        x[i] = (b[i] - sum) / a[i][i];
```

```
    }

    return 0;
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        cout << "использование: " << argv[0] << " <файл_с_матрицей>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if (!in.is_open()) {
        cerr << "ошибка: не удалось открыть файл.\n";
        return 1;
    }

    int n;
    in >> n;

    vector<vector<double>> A(n, vector<double>(n));
    vector<double> b(n);
    vector<double> x;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            in >> A[i][j];
        }
        in >> b[i];
    }
    in.close();

    cout << "размер матрицы: " << n << "x" << n << "\n";
    cout << "решаем систему\n";

    auto start_time = chrono::high_resolution_clock::now();

    int status = GAUSS(A, b, x, n);

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end_time - start_time;

    if (status == 1) {
        cout << "ошибка: Матрица вырождена!\n";
    } else if (status == 2) {
        cout << "ошибка: n < 2\n";
    } else if (status == 3) {
        cout << "ошибка: n > NMAX (" << NMAX << ")\n";
    } else if (status == 0) {
         cout << "система успешно решена за " << elapsed.count() << "
сек.\n";

        double norm_L2 = 0.0;
        double norm_Linf = 0.0;

        for (int i = 0; i < n; ++i) {
            double ax_val = 0.0;
            for (int j = 0; j < n; ++j) {
```

```
                ax_val += A[i][j] * x[j];
            }
            double residual = std::abs(ax_val - b[i]);

            norm_L2 += residual * residual;
            if (residual > norm_Linf) norm_Linf = residual;
        }
        norm_L2 = std::sqrt(norm_L2);

        cout << "\nРЕЗУЛЬТАТЫ\n";
        cout << scientific << setprecision(6);
        cout << "норма невязки L2: " << norm_L2 << "\n";
        cout << "норма невязки L_inf: " << norm_Linf << "\n";

        if (n <= 20) {
            cout << "\nвектор решений x:\n";
            for (int i = 0; i < n; ++i) {
                cout << "x[" << i << "] = " << fixed << setprecision(6)
<< x[i] << "\n";
            }
        } else {
              cout << "\n(матрица слишком большая для вывода вектора
решений)\n";
        }
    }

    return 0;
}
```

Исходный код eigen_solver.cpp:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <eigen3/Eigen/Dense>

using namespace std;
using namespace Eigen;

int main(int argc, char* argv[]) {
    if (argc != 2) {
        cout << "Использование: " << argv[0] << " <файл_с_матрицей>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if (!in.is_open()) {
        cerr << "ошибка: не удалось открыть файл.\n";
        return 1;
    }

    int n;
    in >> n;

    MatrixXd A(n, n);
    VectorXd b(n);

    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
            in >> A(i, j);
        }
        in >> b(i);
    }
    in.close();

    cout << "размер матрицы: " << n << "x" << n << "\n";
    cout << "решаем систему методом Eigen\n";

    auto start_time = chrono::high_resolution_clock::now();

    VectorXd x = A.partialPivLu().solve(b);

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end_time - start_time;

    cout << "Eigen решил систему за " << elapsed.count() << " сек.\n";

    VectorXd residual = A * x - b;
    double norm_L2 = residual.norm();
    double norm_Linf = residual.lpNorm<Infinity>();

    cout << "\nрезультаты EIGEN\n";
    cout << scientific << setprecision(6);
    cout << "норма невязки L2: " << norm_L2 << "\n";
    cout << "норма невязки L_inf: " << norm_Linf << "\n";

    return 0;
}
```

Исходный код generate.cpp:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <random>
#include <cmath>
#include <iomanip>
#include <string>

using namespace std;

void generateRandomDiagonallyDominant(int n, vector<vector<double>>& A,
vector<double>& b) {
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<double> dist(-50.0, 50.0);

    for (int i = 0; i < n; ++i) {
        double sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (i != j) {
                A[i][j] = dist(gen);
                sum += std::abs(A[i][j]);
            }
        }
        A[i][i] = sum + std::abs(dist(gen)) + 1.0;
        if (dist(gen) < 0) A[i][i] = -A[i][i];
```

```
        b[i] = dist(gen);
    }
}

void generateHilbert(int n, vector<vector<double>>& A, vector<double>&
b) {
    for (int i = 0; i < n; ++i) {
        b[i] = 0.0;
        for (int j = 0; j < n; ++j) {
            A[i][j] = 1.0 / (i + j + 1.0);
            b[i] += A[i][j];
        }
    }
}

int main(int argc, char* argv[]) {
    if (argc != 4) {
        cout << "неверные арги\n";
        return 1;
    }

    int n = stoi(argv[1]);
    int type = stoi(argv[2]);
    string filename = argv[3];

    vector<vector<double>> A(n, vector<double>(n));
    vector<double> b(n);

    if (type == 1) {
        generateRandomDiagonallyDominant(n, A, b);
           cout << "сгенерирована случайная матрица с диагональным
преобладанием\n";
    } else if (type == 2) {
        generateHilbert(n, A, b);
        cout << "сгенерирована матрица Гильберта\n";
    } else {
        cout << "введенно неверное значение\n";
        return 1;
    }

    ofstream out(filename);
    if (!out.is_open()) {
        cerr << "ошибка создания файла!\n";
        return 1;
    }

    out << n << "\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            out << fixed << setprecision(10) << A[i][j] << " ";
        }
        out << b[i] << "\n";
    }

    out.close();
    cout << "данные записаны в " << filename << "\n";
    return 0;
```

```
}
```

Исходный код Jacobi.cpp:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <cmath>
#include <algorithm>

using namespace std;

// норма матрицы итераций Якоби ||B||_бесконечность
// B_ij = -a_ij / a_ii для i != j, и 0 на диагонали
double computeJacobiNorm(const vector<vector<double>>& A) {
    int n = A.size();
    double max_sum = 0.0;
    for (int i = 0; i < n; ++i) {
        double diag = fabs(A[i][i]);
        if (diag < 1e-15) return 1e20; // метод не применим
        double sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (j != i) sum += fabs(A[i][j]) / diag;
        }
        if (sum > max_sum) max_sum = sum;
    }
    return max_sum;
}

bool jacobi(const vector<vector<double>>& A, const vector<double>& b,
vector<double>& x, int maxIter, double tol,
                int& iter_count, double& residual_norm_L2, double&
residual_norm_Linf) {

    int n = A.size();

    for (int i = 0; i < n; ++i) {
        if (fabs(A[i][i]) < 1e-15) {
            cerr << "ошибка: нулевой диагональный элемент, метод Якоби
неприменим\n";
            return false;
        }
    }


    double normB = computeJacobiNorm(A);
    cout << "норма матрицы итераций Якоби = " << normB << "\n";
    if (normB >= 1.0) {
        cout << "предупреждение: ||B|| >= 1, метод может расходиться или
сходиться очень медленно\n";
    }

    x.assign(n, 0.0);
    vector<double> x_new(n, 0.0);

    for (iter_count = 0; iter_count < maxIter; ++iter_count) {
```

```
        // итерация метода
        for (int i = 0; i < n; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n; ++j) {
                 if (j != i) sum += A[i][j] * x[j]; // часть формулы,
считаем сумму всех элементов матрицы умноженных на вектор приближение
            }
            x_new[i] = (b[i] - sum) / A[i][i]; // новое приблежиние
        }

        // считаем евкидову норму
        // суть -- с каждой итерацией мы находим все большее приблежение
        // соответсвенно разница между приближениями должна уменьшаться
         // при достижение достаточно маленькой разности можно считать
что дальше итерировать нет смыысла решение найдено
        double err = 0.0;
        for (int i = 0; i < n; ++i) {
            double diff = x_new[i] - x[i];
            err += diff * diff;
        }
        err = sqrt(err); // ( (сумма всех x_new[i] - x[i])^2 )^1/2

        x = x_new;

          // если нормал2 меньше заданной погрешности мы заканчиваем
итерации и возвращаем нынешенее приблежиние как решение
        if (err < tol) {
            // невязки
            residual_norm_L2 = 0.0;
            residual_norm_Linf = 0.0;
            for (int i = 0; i < n; ++i) {
                double Ax = 0.0;
                 for (int j = 0; j < n; ++j) Ax += A[i][j] * x[j]; //
скалярное произведение iй строки матрицы A на вектор x
                double resid = Ax - b[i]; // iтая компонента невязки
                residual_norm_L2 += resid * resid;
               if (fabs(resid) > residual_norm_Linf) residual_norm_Linf
= fabs(resid);
            }
            residual_norm_L2 = sqrt(residual_norm_L2);
            return true;
        }
    }

    // если дошли сюда -- не сошлись за максчисло итераций итераций
    // посчитаем невязку для текущего x
    residual_norm_L2 = 0.0;
    residual_norm_Linf = 0.0;
    for (int i = 0; i < n; ++i) {
        double Ax = 0.0;
        for (int j = 0; j < n; ++j) Ax += A[i][j] * x[j];
        double resid = Ax - b[i];
        residual_norm_L2 += resid * resid;
          if (fabs(resid) > residual_norm_Linf) residual_norm_Linf =
fabs(resid);
    }
    residual_norm_L2 = sqrt(residual_norm_L2);
    return false;
```

```
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        cout << "гайд: " << argv[0] << " <файл_с_матрицей>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if (!in.is_open()) {
        cerr << "ошибка: не удалось открыть файл.\n";
        return 1;
    }

    int n;
    in >> n;

    vector<vector<double>> A(n, vector<double>(n));
    vector<double> b(n);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            in >> A[i][j];
        }
        in >> b[i];
    }
    in.close();

    cout << "размер матрицы: " << n << "x" << n << "\n";
    cout << "решение системы методом Якоби (простых итераций)\n";

    const int maxIter = 100000;
    const double tol = 1e-12;

    vector<double> x;
    int iter_count;
    double residual_L2, residual_Linf;

    auto start_time = chrono::high_resolution_clock::now();

      bool converged = jacobi(A, b, x, maxIter, tol, iter_count,
residual_L2, residual_Linf);

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end_time - start_time;

    if (converged) {
        cout << "метод сошёлся за " << iter_count << " итераций.\n";
    } else {
        cout << "метод не сошёлся за " << maxIter << " итераций.\n";
    }
    cout << "время выполнения: " << elapsed.count() << " сек.\n";

    cout << "\nРЕЗУЛЬТАТЫ\n";
    cout << scientific << setprecision(6);
    cout << "Норма невязки L2:   " << residual_L2 << "\n";
    cout << "Норма невязки L_inf: " << residual_Linf << "\n";
```

```
    // первые 10 компонент решения (матрица не слишком большая)
    if (n <= 20) {
        cout << "\nвектор решения x:\n";
        cout << fixed << setprecision(8);
        for (int i = 0; i < n; ++i) {
            cout << "x[" << i << "] = " << x[i] << "\n";
        }
    } else {
        cout << "\n(размер матрицы > 20, вывод решения опущен)\n";
        cout << "первые 5 компонент: ";
        for (int i = 0; i < min(5, n); ++i) cout << x[i] << " ";
        cout << "\n";
    }

    return 0;
}
```

Исходный код full.cpp:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <cmath>
#include <algorithm>
#include <eigen3/Eigen/Dense>

using namespace std;
using namespace Eigen;

const int NMAX = 5000;

void computeResidual(const vector<vector<double>>& A, const vector<double>& b,
                     const vector<double>& x, double& l2, double& linf)
{
    int n = A.size();
    l2 = 0.0;
    linf = 0.0;
    for (int i = 0; i < n; ++i) {
        double Ax = 0.0;
        for (int j = 0; j < n; ++j) Ax += A[i][j] * x[j];
        double resid = Ax - b[i];
        l2 += resid * resid;
        if (fabs(resid) > linf) linf = fabs(resid);
    }
    l2 = sqrt(l2);
}

void computeDiff(const vector<double>& x, const vector<double>& y,
                 double& diff_l2, double& diff_linf) {
    int n = x.size();
    diff_l2 = 0.0;
    diff_linf = 0.0;
    for (int i = 0; i < n; ++i) {
        double d = x[i] - y[i];
        diff_l2 += d * d;
        if (fabs(d) > diff_linf) diff_linf = fabs(d);
    }
```

```
    diff_l2 = sqrt(diff_l2);
}

int GAUSS(const vector<vector<double>>& a_in, const vector<double>& b_in,
vector<double>& x, int n) {
    if (n < 2) return 2;
    if (n > NMAX) return 3;

    vector<vector<double>> a = a_in;
    vector<double> b = b_in;
    x.assign(n, 0.0);

    for (int i = 0; i < n; ++i) {

        int pivot_row = i;
        double max_val = std::abs(a[i][i]);

        for (int k = i + 1; k < n; ++k) {
            if (std::abs(a[k][i]) > max_val) {
                max_val = std::abs(a[k][i]);
                pivot_row = k;
            }
        }

        if (max_val < 1e-12) {
            return 1;
        }

        if (pivot_row != i) {
            swap(a[i], a[pivot_row]);
            swap(b[i], b[pivot_row]);
        }

        for (int k = i + 1; k < n; ++k) {
            double factor = a[k][i] / a[i][i];
            for (int j = i; j < n; ++j) {
                a[k][j] -= factor * a[i][j];
            }
            b[k] -= factor * b[i];
        }
    }

    for (int i = n - 1; i >= 0; --i) {
        double sum = 0.0;
        for (int j = i + 1; j < n; ++j) {
            sum += a[i][j] * x[j];
        }
        x[i] = (b[i] - sum) / a[i][i];
    }

    return 0;
}

// норма матрицы итераций Якоби ||B||_бесконечность
// B_ij = -a_ij / a_ii для i != j, и 0 на диагонали
double computeJacobiNorm(const vector<vector<double>>& A) {
    int n = A.size();
    double max_sum = 0.0;
```

```
    for (int i = 0; i < n; ++i) {
        double diag = fabs(A[i][i]);
        if (diag < 1e-15) return 1e20; // метод не применим
        double sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (j != i) sum += fabs(A[i][j]) / diag;
        }
        if (sum > max_sum) max_sum = sum;
    }
    return max_sum;
}

bool jacobi(const vector<vector<double>>& A, const vector<double>& b,
vector<double>& x, int maxIter, double tol,
                int& iter_count, double& residual_norm_L2, double&
residual_norm_Linf) {

    int n = A.size();

    for (int i = 0; i < n; ++i) {
        if (fabs(A[i][i]) < 1e-15) {
            cerr << "ошибка: нулевой диагональный элемент, метод Якоби
неприменим\n";
            return false;
        }
    }


    double normB = computeJacobiNorm(A);
    cout << "норма матрицы итераций Якоби = " << normB << "\n";
    if (normB >= 1.0) {
       cout << "предупреждение: ||B|| >= 1, метод может расходиться или
сходиться очень медленно\n";
    }

    x.assign(n, 0.0);
    vector<double> x_new(n, 0.0);

    for (iter_count = 0; iter_count < maxIter; ++iter_count) {

        // итерация метода
        for (int i = 0; i < n; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n; ++j) {
                 if (j != i) sum += A[i][j] * x[j]; // часть формулы,
считаем сумму всех элементов матрицы умноженных на вектор приближение
            }
            x_new[i] = (b[i] - sum) / A[i][i]; // новое приблежиние
        }

        // считаем евкидову норму
        // суть -- с каждой итерацией мы находим все большее приблежение
        // соответсвенно разница между приближениями должна уменьшаться
         // при достижение достаточно маленькой разности можно считать
что дальше итерировать нет смыысла решение найдено
        double err = 0.0;
        for (int i = 0; i < n; ++i) {
            double diff = x_new[i] - x[i];
```

```
            err += diff * diff;
        }
        err = sqrt(err); // ( (сумма всех x_new[i] - x[i])^2 )^1/2

        x = x_new;

          // если нормал2 меньше заданной погрешности мы заканчиваем
итерации и возвращаем нынешенее приблежиние как решение
        if (err < tol) {
            // невязки
            residual_norm_L2 = 0.0;
            residual_norm_Linf = 0.0;
            for (int i = 0; i < n; ++i) {
                double Ax = 0.0;
                 for (int j = 0; j < n; ++j) Ax += A[i][j] * x[j]; //
скалярное произведение iй строки матрицы A на вектор x
                double resid = Ax - b[i]; // iтая компонента невязки
                residual_norm_L2 += resid * resid;
               if (fabs(resid) > residual_norm_Linf) residual_norm_Linf
= fabs(resid);
            }
            residual_norm_L2 = sqrt(residual_norm_L2);
            return true;
        }
    }

    // если дошли сюда -- не сошлись за максчисло итераций итераций
    // посчитаем невязку для текущего x
    residual_norm_L2 = 0.0;
    residual_norm_Linf = 0.0;
    for (int i = 0; i < n; ++i) {
        double Ax = 0.0;
        for (int j = 0; j < n; ++j) Ax += A[i][j] * x[j];
        double resid = Ax - b[i];
        residual_norm_L2 += resid * resid;
          if (fabs(resid) > residual_norm_Linf) residual_norm_Linf =
fabs(resid);
    }
    residual_norm_L2 = sqrt(residual_norm_L2);
    return false;
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        cerr << "использование: " << argv[0] << " <файл_с_матрицей>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if (!in.is_open()) {
        cerr << "ошибка: не удалось открыть файл " << argv[1] << endl;
        return 1;
    }

    int n;
    in >> n;
    if (n <= 0) {
        cerr << "ошибка: неверный размер матрицы." << endl;
```

```
        return 1;
    }

    vector<vector<double>> A(n, vector<double>(n));
    vector<double> b(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) in >> A[i][j];
        in >> b[i];
    }
    in.close();

    cout << fixed << setprecision(12);
    cout << "размер системы: " << n << "x" << n << "\n\n";


    // 1. метод Гаусс
    vector<double> x_gauss;
    auto start = chrono::high_resolution_clock::now();
    int gauss_status = GAUSS(A, b, x_gauss, n);
    auto end = chrono::high_resolution_clock::now();
    double time_gauss = chrono::duration<double>(end - start).count();

    double l2_gauss, linf_gauss;
    if (gauss_status == 0) {
        computeResidual(A, b, x_gauss, l2_gauss, linf_gauss);
    }


    // 2. метод Якоби
    vector<double> x_jacobi;
    int iter_count;
    double l2_jacobi, linf_jacobi;
    bool jacobi_ok = false;
    double time_jacobi = 0.0;
    if (gauss_status != 1) {
        start = chrono::high_resolution_clock::now();
         jacobi_ok = jacobi(A, b, x_jacobi, 100000, 1e-12, iter_count,
l2_jacobi, linf_jacobi);
        end = chrono::high_resolution_clock::now();
        time_jacobi = chrono::duration<double>(end - start).count();
    }


    // 3. метод Eigen (PartialPivLU)

    MatrixXd A_eig(n, n);
    VectorXd b_eig(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) A_eig(i, j) = A[i][j];
        b_eig(i) = b[i];
    }
    start = chrono::high_resolution_clock::now();
    VectorXd x_eig = A_eig.partialPivLu().solve(b_eig);
    end = chrono::high_resolution_clock::now();
    double time_eigen = chrono::duration<double>(end - start).count();
    VectorXd resid_eig = A_eig * x_eig - b_eig;
    double l2_eigen = resid_eig.norm();
    double linf_eigen = resid_eig.lpNorm<Infinity>();
```

```
    vector<double> x_eigen_vec(x_eig.data(), x_eig.data() + n);


    cout << "метод ГАУССА\n";
    if (gauss_status == 0) {
        cout << "Время: " << time_gauss << " сек\n";
        cout << "Невязка L2   : " << scientific << l2_gauss << "\n";
        cout << "Невязка L_inf   : " << linf_gauss << "\n";
    } else if (gauss_status == 1) {
       cout << "ошибка: матрица вырождена (метод Гаусса неприменим)\n";
    } else {
         cout << "ошибка: неверный размер или превышен лимит (n<2 или
n>NMAX)\n";
    }

    cout << "\nметод ЯКОБИ\n";
    if (jacobi_ok) {
        cout << "Сошёлся за " << iter_count << " итераций\n";
        cout << "Время: " << time_jacobi << " сек\n";
        cout << "Невязка L2   : " << scientific << l2_jacobi << "\n";
        cout << "Невязка L_inf   : " << linf_jacobi << "\n";
    } else {
         cout << "метод Якоби не сошёлся или неприменим (диагональные
элементы близки к нулю)\n";
       if (iter_count > 0) cout << "последняя невязка L2: " << l2_jacobi
<< "\n";
    }

    cout << "\nметод EIGEN\n";
    cout << "Время: " << time_eigen << " сек\n";
    cout << "Невязка L2   : " << scientific << l2_eigen << "\n";
    cout << "Невязка L_inf   : " << linf_eigen << "\n";


    cout << "\nПОПАРНЫЕ РАЗНОСТИ РЕШЕНИЙ\n";

    if (gauss_status == 0 && jacobi_ok) {
        double diff_l2, diff_linf;
        computeDiff(x_gauss, x_jacobi, diff_l2, diff_linf);
        cout << "||Gauss - Jacobi||_2  = " << diff_l2 << "\n";
        cout << "||Gauss - Jacobi||_inf  = " << diff_linf << "\n";
    }
    if (gauss_status == 0) {
        double diff_l2, diff_linf;
        computeDiff(x_gauss, x_eigen_vec, diff_l2, diff_linf);
        cout << "||Gauss - Eigen||_2   = " << diff_l2 << "\n";
        cout << "||Gauss - Eigen||_inf   = " << diff_linf << "\n";
    }
    if (jacobi_ok) {
        double diff_l2, diff_linf;
        computeDiff(x_jacobi, x_eigen_vec, diff_l2, diff_linf);
        cout << "||Jacobi - Eigen||_2  = " << diff_l2 << "\n";
        cout << "||Jacobi - Eigen||_inf  = " << diff_linf << "\n";
    }

    if (gauss_status == 1 && !jacobi_ok) {
       cout << "матрица, вероятно, вырождена или плохо обусловлена.\n";
        cout << "результаты сравнения могут быть ненадёжными.\n";
```

```
    }

    return 0;
}
```